BUILD / TEST / EXPLAIN

AWS

VPC Peering

Connect two VPCs.
Prove private connectivity.

Project report: build, test and explain a two-VPC peering lab with a bastion host, custom route tables and scoped security groups.

2 VPCs / 3 EC2 INSTANCES / 1 PRIVATE CONNECTION

AWS ARCHITECT ASSOCIATE

Name	**Pratiksha Hiremath**
Batch	_____
Project	**Project 01 - AWS VPC Peering**
Region	**Asia Pacific (Mumbai) / ap-south-1**
Completion date	**26 October 2026**

Contents

1. Architecture and resource inventory . . . 3

2. Network build (Checkpoint 01) . . . 5

3. Security groups and instances (Checkpoint 02) . . . 10

4. Administrative access with SSH ProxyJump . . . 14

5. Peering connection and routes (Checkpoints 03 and 04) . . . 15

6. Verification tests . . . 17

7. Explanations . . . 20

8. Cleanup evidence . . . 21

9. What I learned . . . 24

Project summary

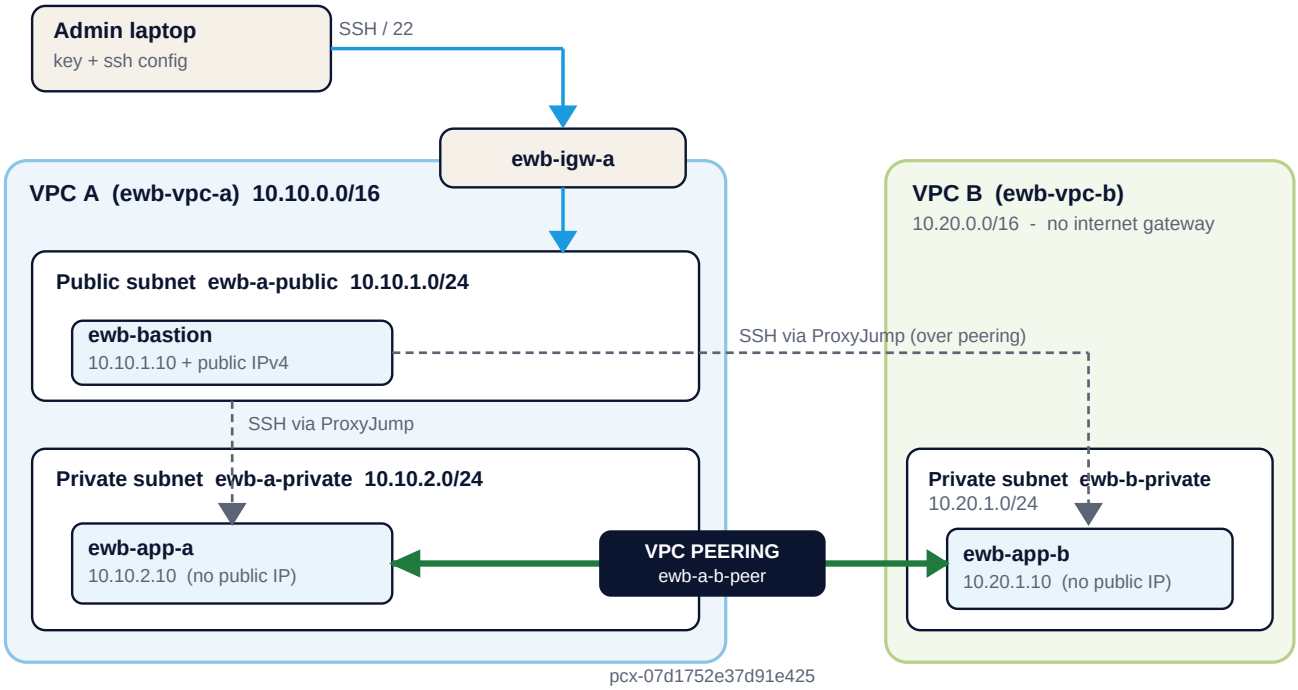
EWB has an application in VPC A and a second workload in VPC B. Both application servers must stay private. This project connects the two VPCs with a VPC peering connection so the servers can exchange traffic using their private IPv4 addresses, and uses one public bastion host in VPC A for administration. Everything was built in a single AWS account and a single Region (Mumbai, ap-south-1) using Amazon Linux 2023 on t3.micro instances, then tested and removed. No NAT gateway was used.

What this report demonstrates

- Non-overlapping VPC and subnet CIDRs (10.10.0.0/16 and 10.20.0.0/16).
- Public and private subnets with explicit route table associations.
- Peering activated, forward and return routes added, and only the required inbound traffic allowed (single-host /32 rules).
- A failed test before peering, successful tests after peering, and a controlled failure with recovery.
- Cleanup of every resource, with evidence.

1. Architecture and resource inventory

One internet entry point (the bastion in VPC A's public subnet), two private workloads, and a direct peering connection between the VPCs. Administration path: laptop to bastion to App-A or App-B using SSH ProxyJump. Project traffic: App-A and App-B talk over their private IPs and the peering connection. VPC B has no internet gateway, and peering does not give it internet access through VPC A.



Address plan

Resource / Name tag	VPC or subnet	IPv4 CIDR / address
ewb-vpc-a	VPC A	10.10.0.0/16
ewb-a-public	VPC A / public subnet	10.10.1.0/24
ewb-a-private	VPC A / private subnet	10.10.2.0/24
ewb-vpc-b	VPC B	10.20.0.0/16
ewb-b-private	VPC B / private subnet	10.20.1.0/24
ewb-bastion	ewb-a-public	10.10.1.10 + public IPv4
ewb-app-a	ewb-a-private	10.10.2.10 / no public IP
ewb-app-b	ewb-b-private	10.20.1.10 / no public IP

Resource inventory (actual IDs)

Resource	Name	ID	Details
VPC	ewb-vpc-a	vpc-06e720377d5ccfde4	10.10.0.0/16
VPC	ewb-vpc-b	vpc-02f37992d1005c36e	10.20.0.0/16
Subnet	ewb-a-public	subnet-0ca80f051e85c2a7b	10.10.1.0/24, ap-south-1a
Subnet	ewb-a-private	subnet-06099a0f447ebbc87	10.10.2.0/24, ap-south-1a
Subnet	ewb-b-private	subnet-05d0b6f5809f5bcb1	10.20.1.0/24, ap-south-1a
Internet gateway	ewb-igw-a	igw-0f004578e06527e50	Attached to ewb-vpc-a
Route table	ewb-rt-a-public	rtb-0895958b3803cb05d	ewb-a-public
Route table	ewb-rt-a-private	rtb-0fd680bac249ef42a	ewb-a-private
Route table	ewb-rt-b-private	rtb-015efbf3b3bfc41a3	ewb-b-private
Peering	ewb-a-b-peer	pcx-07d1752e37d91e425	ewb-vpc-a (requester) to ewb-vpc-b (accepter)
Security group	ewb-sg-bastion	sg-03fca31f3c336cca9	VPC A
Security group	ewb-sg-app-a	sg-071d92cb734b0009b	VPC A
Security group	ewb-sg-app-b	sg-0ef10d26200298383	VPC B
EC2 instance	ewb-bastion	i-02503cc43669b8a52	10.10.1.10, public IPv4 13.233.107.126
EC2 instance	ewb-app-a	i-0ad72fcc422a2b9f0	10.10.2.10, no public IPv4
EC2 instance	ewb-app-b	i-0b9f82a55d12d9af8	10.20.1.10, no public IPv4

All three instances: Amazon Linux 2023 (ami-04394a3a96907e9ec), t3.micro, Availability Zone ap-south-1a, IMDSv2 required, key pair ewb-lab. The bastion public address was auto-assigned and was released when the instance was terminated. The Project and Student tags were added to the lab resources (VPCs, subnets, route tables, internet gateway and instances) using Tag Editor.

2. Network build (Checkpoint 01)

VPC A was created with 10.10.0.0/16 and VPC B with 10.20.0.0/16. The two ranges do not overlap, which is required before AWS will create a peering connection. Each /24 subnet sits inside its own VPC range.

Figure 1. Both lab VPCs created with non-overlapping CIDRs

Host / console: VPC console / Your VPCs | **Result: ewb-vpc-a 10.10.0.0/16, ewb-vpc-b 10.20.0.0/16 (plus the default VPC)**

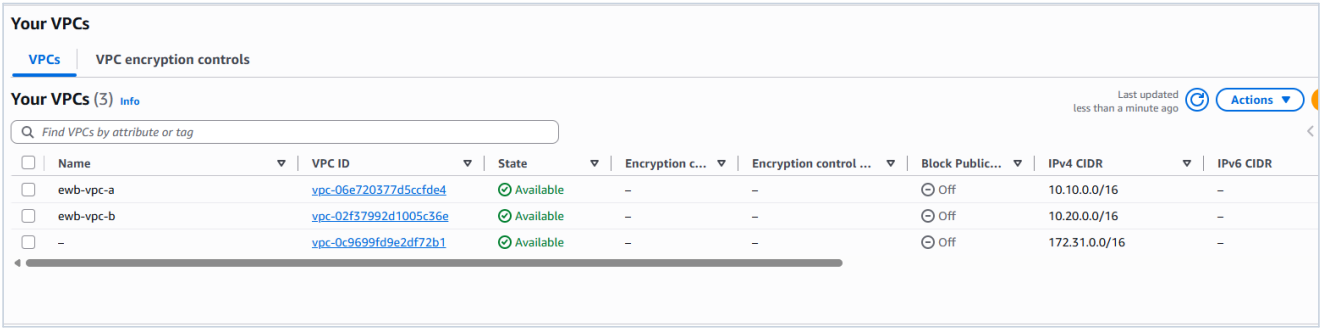


Figure 2. VPC A resource map: two subnets, three route tables, internet gateway

Host / console: VPC console / ewb-vpc-a | **Result: 2 subnets, route tables with associations, ewb-igw-a**

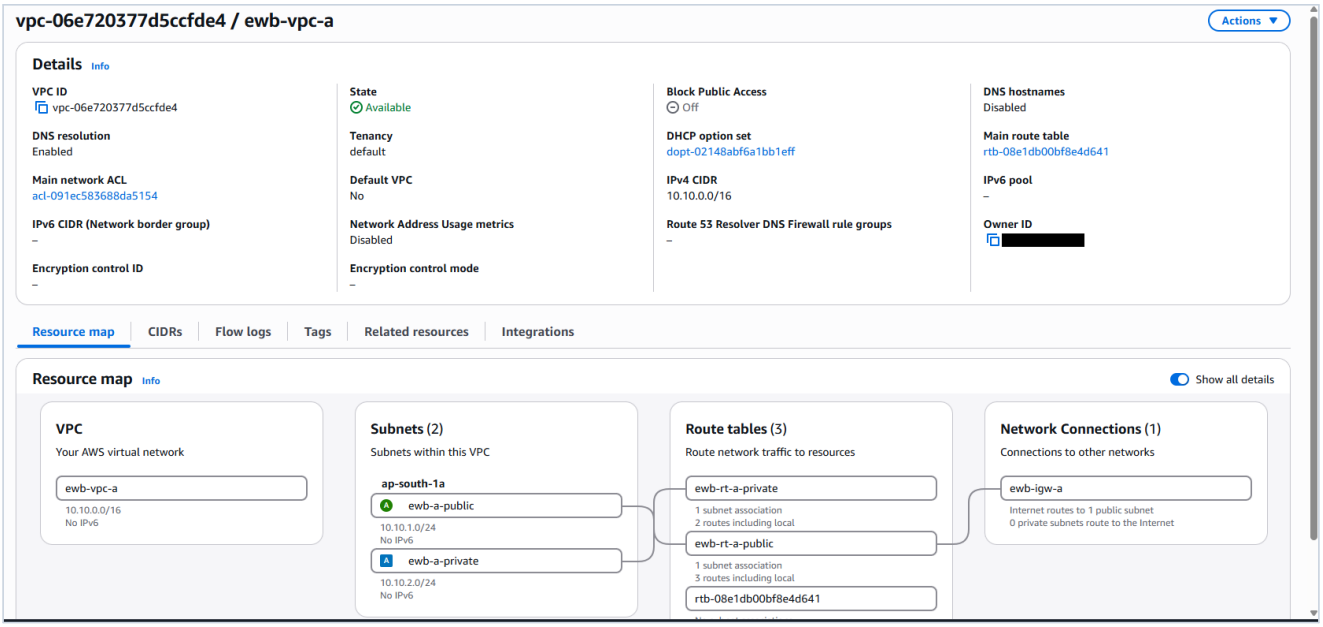
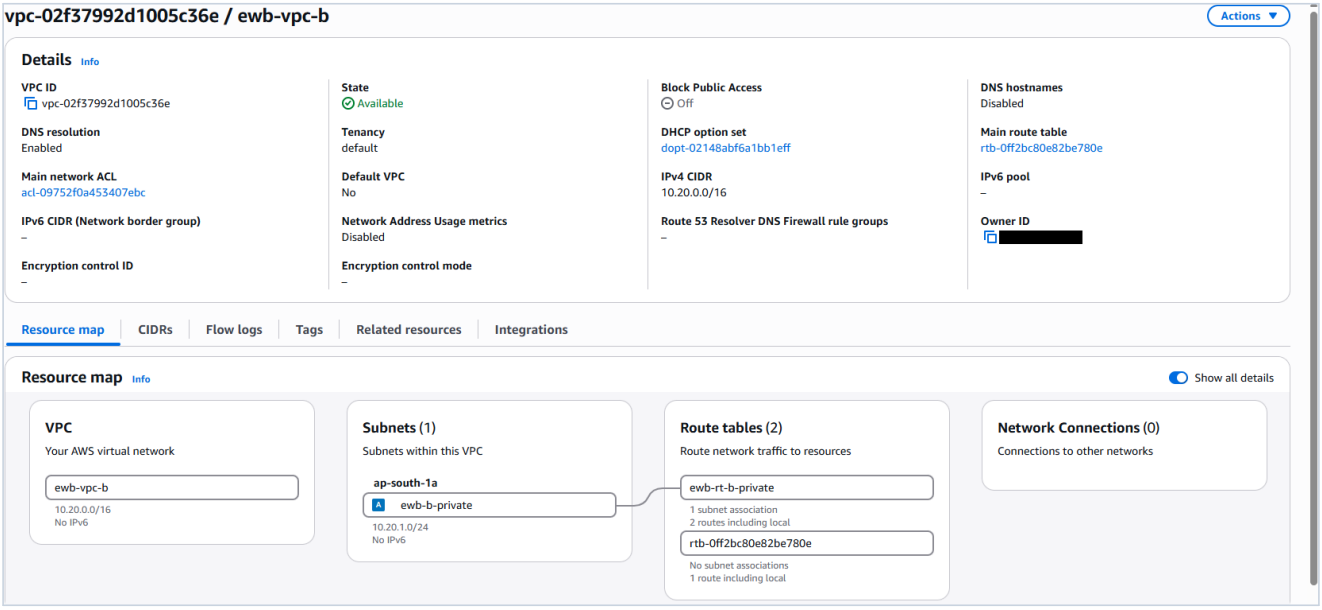


Figure 3. VPC B resource map: one private subnet, no internet gateway

Host / console: VPC console / ewb-vpc-b | **Result: 1 subnet, 0 network connections (no IGW)**



Subnets

Three subnets were created, all in ap-south-1a. Auto-assign public IPv4 is off on every subnet; the bastion receives its public address through its own launch setting.

Figure 4. Subnet list with CIDRs

Host / console: VPC console / Subnets | **Result: 10.10.1.0/24, 10.10.2.0/24, 10.20.1.0/24**

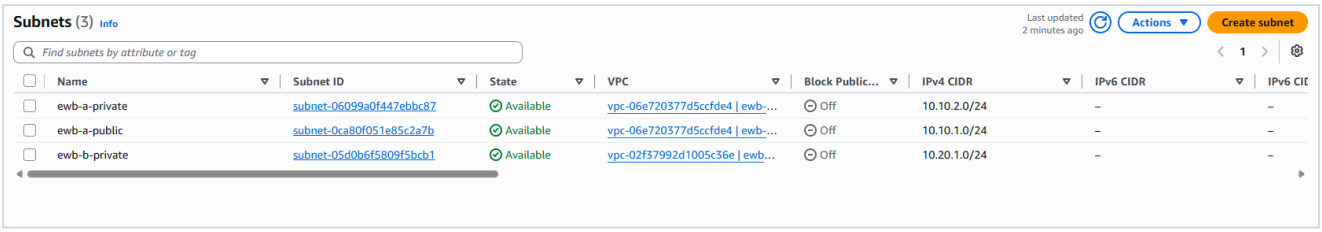


Figure 5. Public subnet ewb-a-public uses route table ewb-rt-a-public

Host / console: VPC console / ewb-a-public | **Result: Route table = ewb-rt-a-public**

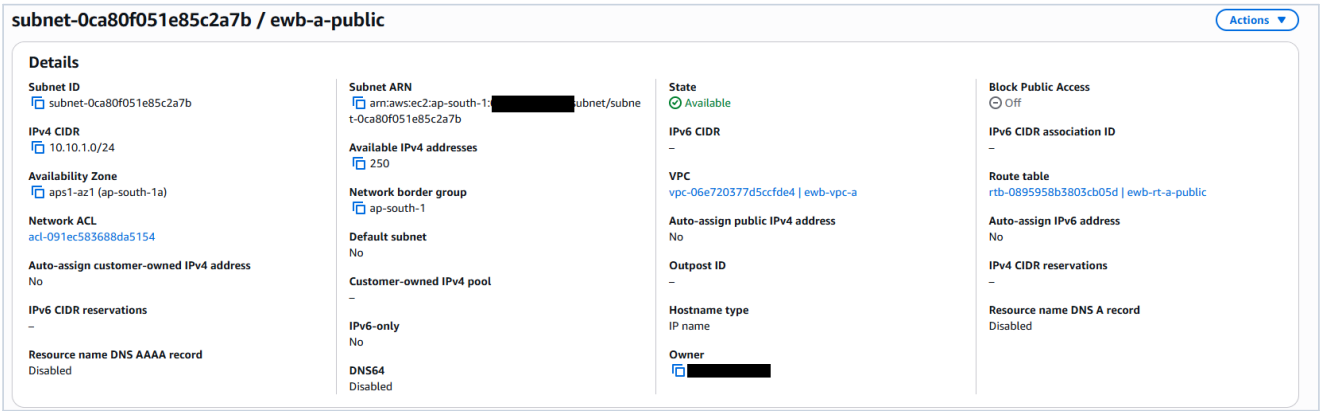


Figure 6. Private subnet ewb-a-private uses route table ewb-rt-a-private
Host / console: VPC console / ewb-a-private | **Result:** **Route table = ewb-rt-a-private**

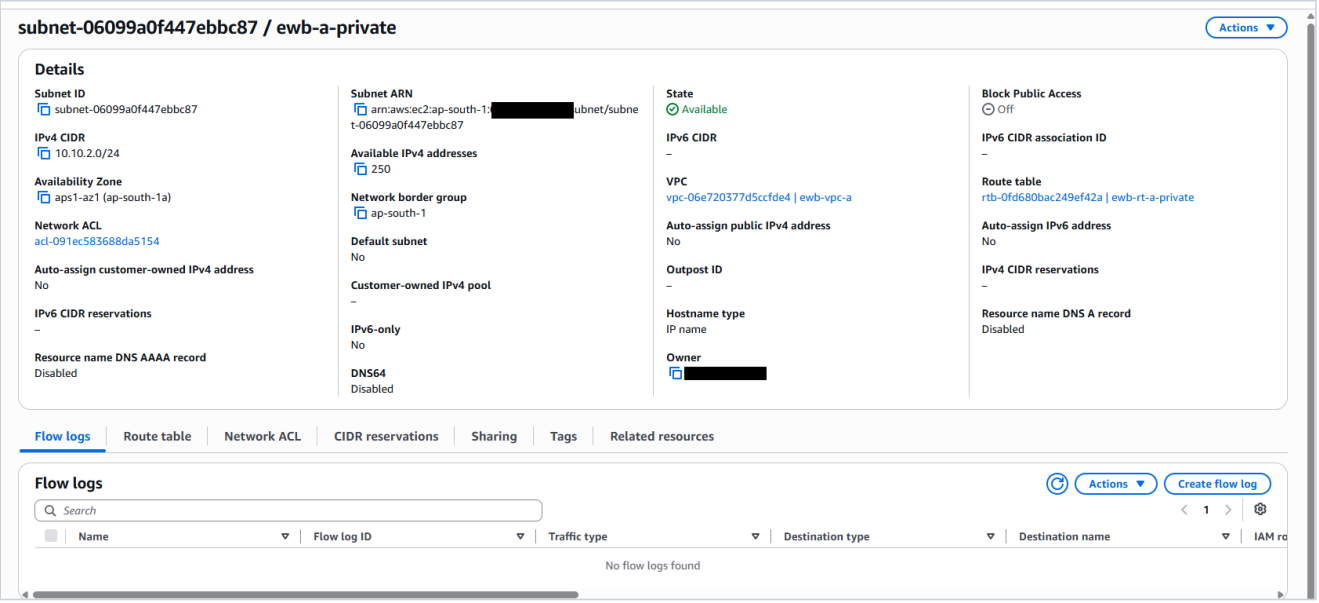
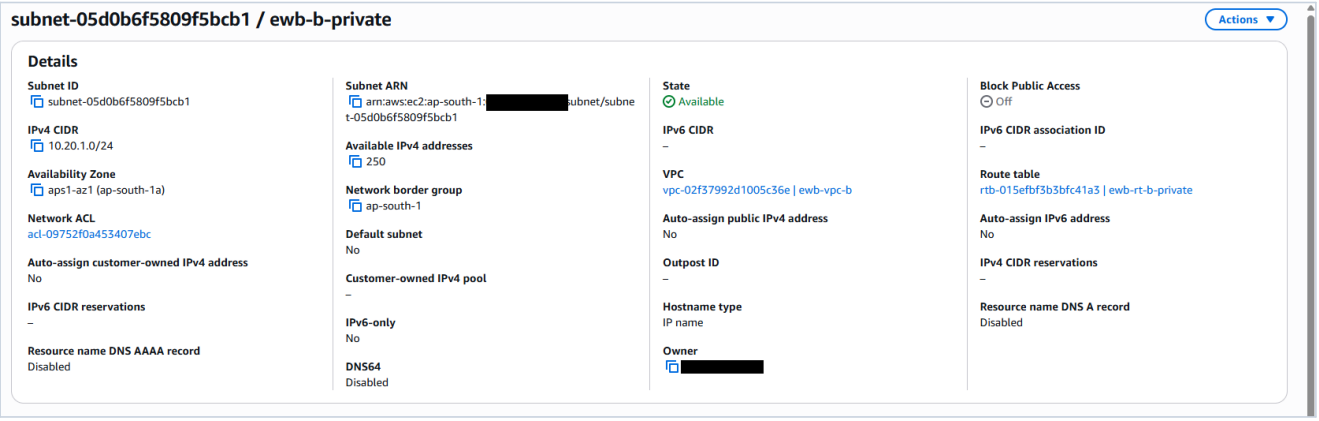


Figure 7. Private subnet ewb-b-private (VPC B) uses route table ewb-rt-b-private
Host / console: VPC console / ewb-b-private | **Result:** **Route table = ewb-rt-b-private**



Route tables before peering

Before the peering connection existed, the public route table held the local route and the internet default route; both private route tables held only their local VPC route. These screenshots are the Checkpoint 01 baseline.

Figure 8. ewb-rt-a-public: subnet association

Host / console: VPC console / ewb-rt-a-public | **Result:** **ewb-a-public explicitly associated**

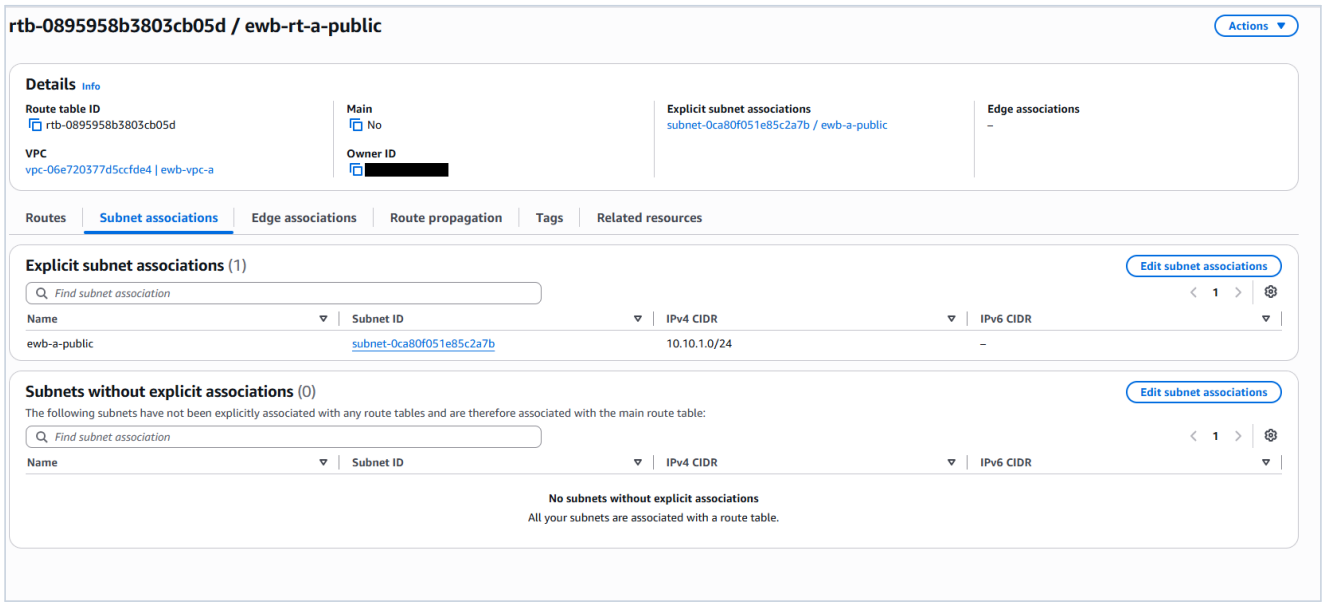


Figure 9. ewb-rt-a-public routes before peering

Host / console: VPC console / ewb-rt-a-public | **Result:** **10.10.0.0/16 local; 0.0.0.0/0 to ewb-igw-a**

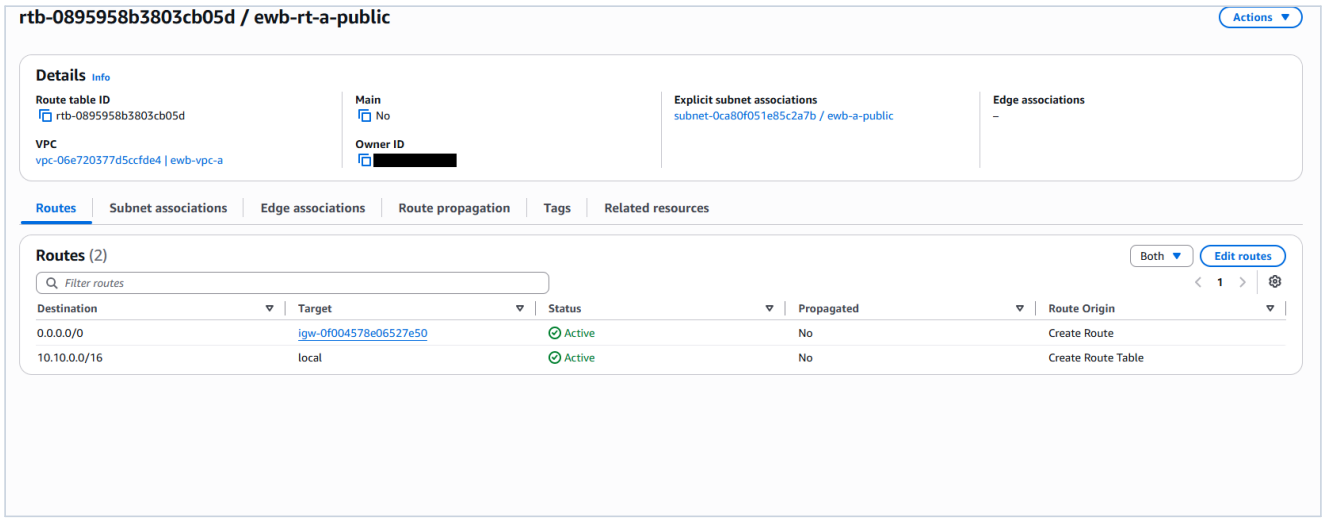


Figure 10. ewb-rt-a-private routes before peering

Host / console: VPC console / ewb-rt-a-private | **Result:** **Only 10.10.0.0/16 local; ewb-a-private associated**

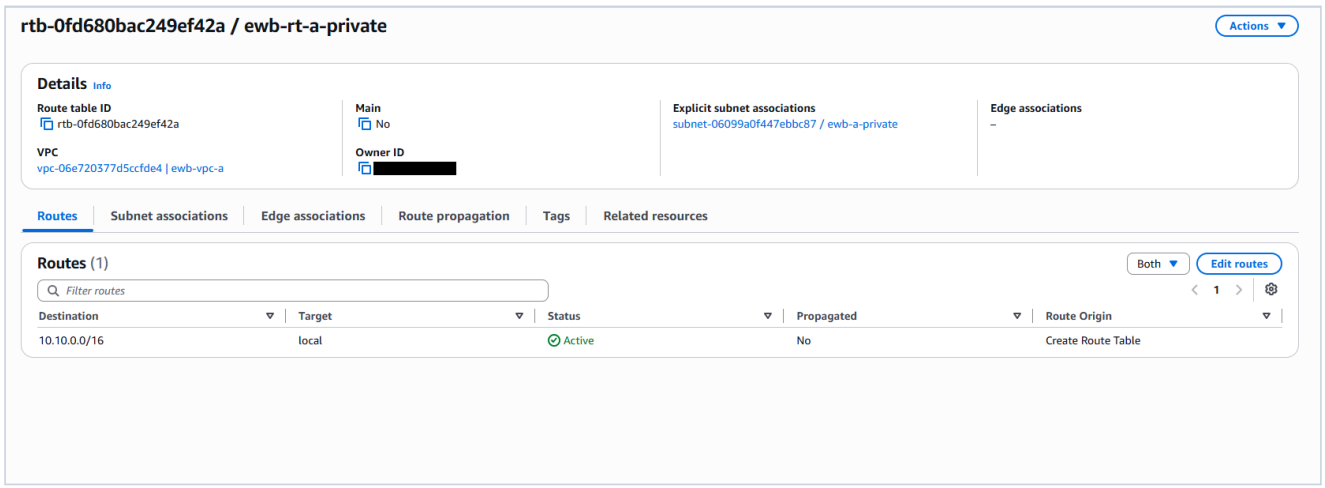


Figure 11. ewb-rt-b-private routes before peering

Host / console: VPC console / ewb-rt-b-private | **Result:** **Only 10.20.0.0/16 local; ewb-b-private associated**

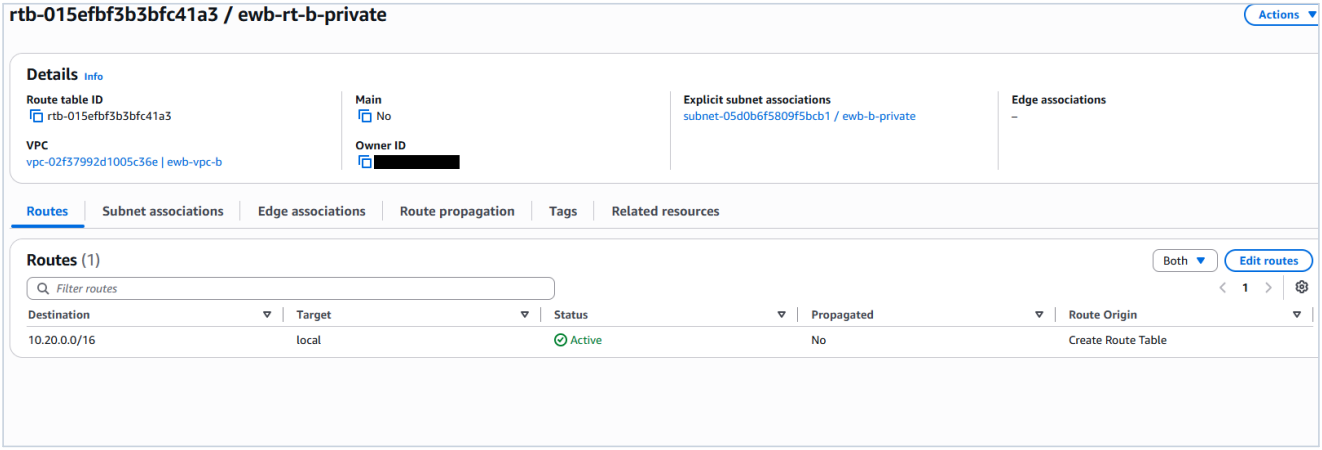
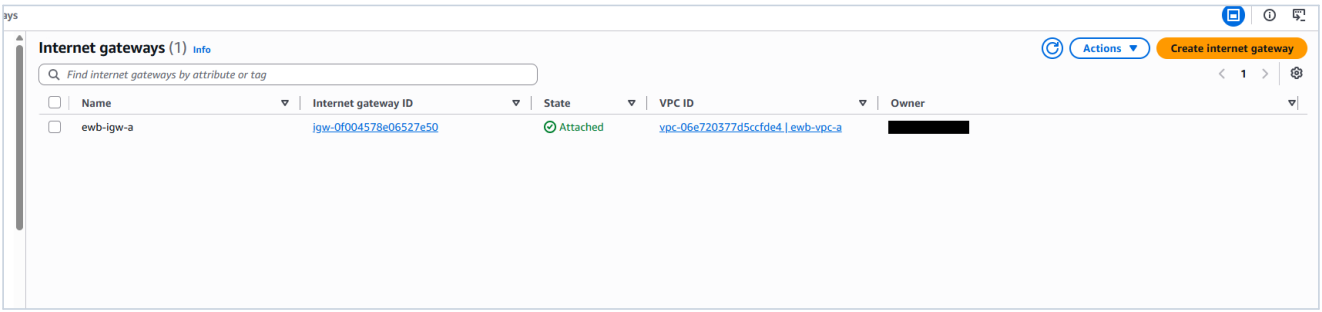


Figure 12. Internet gateway ewb-igw-a

Host / console: VPC console / Internet gateways | **Result:** **State = Attached to ewb-vpc-a**



3. Security groups and instances (Checkpoint 02)

Each security group allows only what the lab needs, and every source is a single host (/32). SSH to the private servers is allowed only from the bastion's private IP; ping (ICMP Echo Request, type 8) is allowed only from the other application server. Outbound traffic keeps the default allow-all rule, and the default network ACLs were not changed.

Security group (VPC)	Direction	Type / port	Source or destination
ewb-sg-bastion (VPC A)	Inbound	SSH / TCP 22	Administrator's public IPv4 /32
ewb-sg-app-a (VPC A)	Inbound	SSH / TCP 22	10.10.1.10/32 (bastion private IP)
ewb-sg-app-a (VPC A)	Inbound	Custom ICMP, Echo Request	10.20.1.10/32 (App-B)
ewb-sg-app-b (VPC B)	Inbound	SSH / TCP 22	10.10.1.10/32 (bastion private IP)
ewb-sg-app-b (VPC B)	Inbound	Custom ICMP, Echo Request	10.10.2.10/32 (App-A)
All three groups	Outbound	All traffic	0.0.0.0/0 (default allow-all)

Figure 13. Inbound rules: ewb-sg-bastion

Host / console: VPC console / Security groups / ewb-sg-bastion | **Result: 1 rule: SSH 22 from one /32 address**

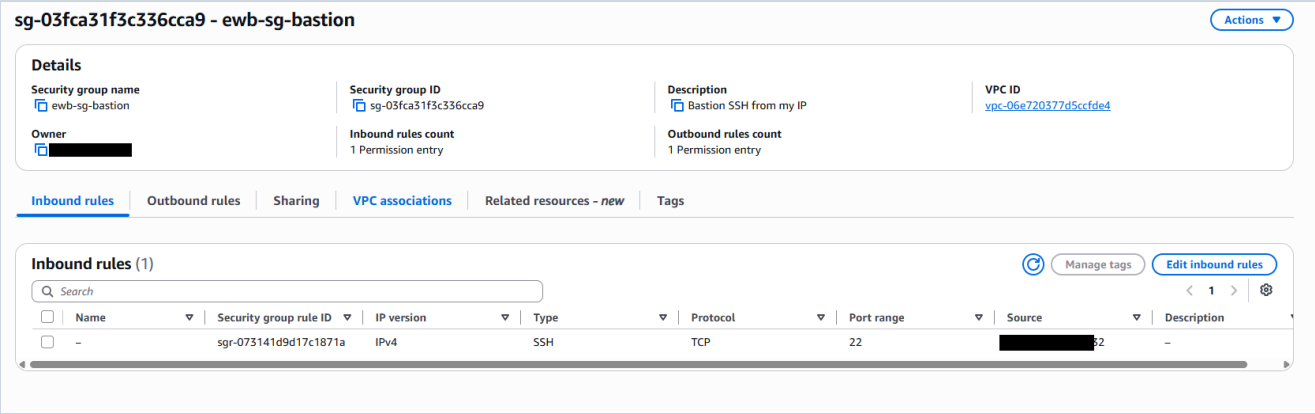


Figure 14. Inbound rules: ewb-sg-app-a

Host / console: VPC console / Security groups / ewb-sg-app-a | **Result: Echo Request from 10.20.1.10/32; SSH 22 from 10.10.1.10/32**

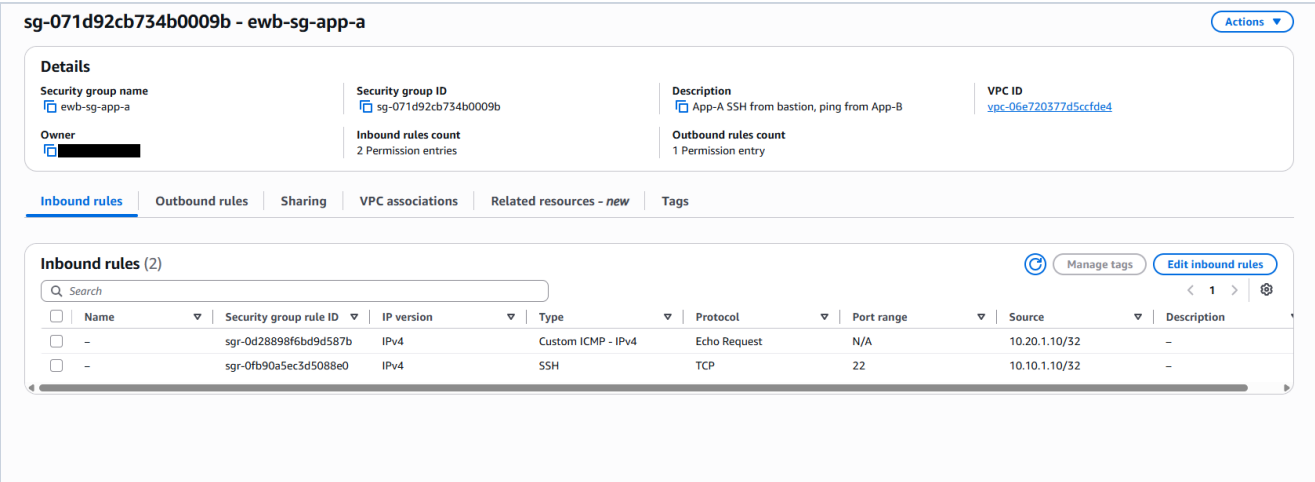


Figure 15. Inbound rules: ewb-sg-app-b

Host / console: VPC console / Security groups / ewb-sg-app-b | **Result: SSH 22 from 10.10.1.10/32; Echo Request from 10.10.2.10/32**

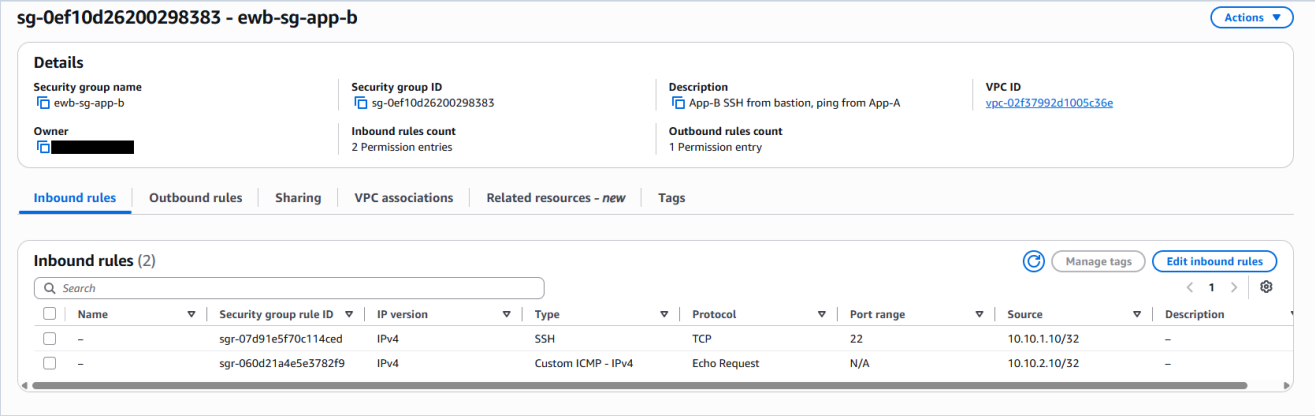


Figure 16. Outbound rules: ewb-sg-bastion

Host / console: VPC console / Security groups / ewb-sg-bastion | **Result: All traffic to 0.0.0.0/0**

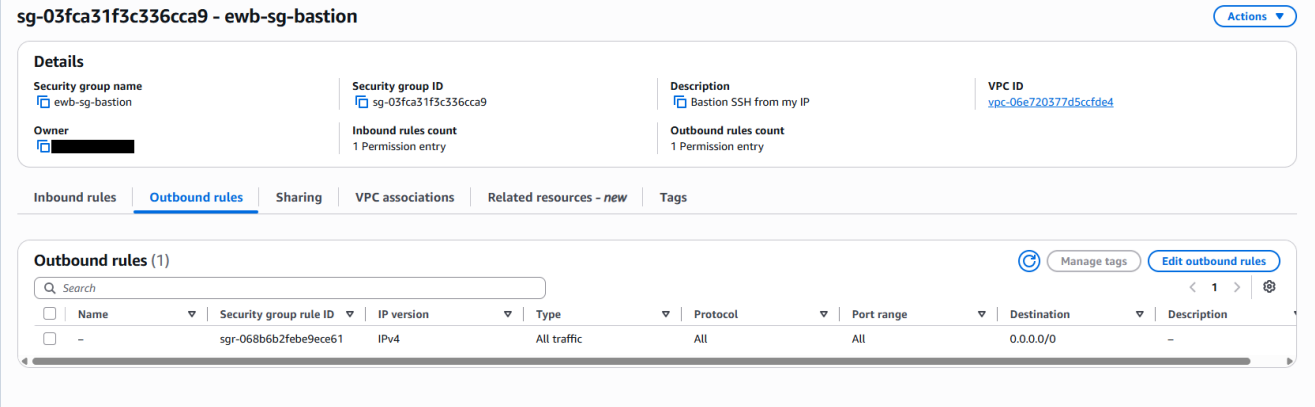


Figure 17. Outbound rules: ewb-sg-app-a

Host / console: VPC console / Security groups / ewb-sg-app-a | **Result: All traffic to 0.0.0.0/0**

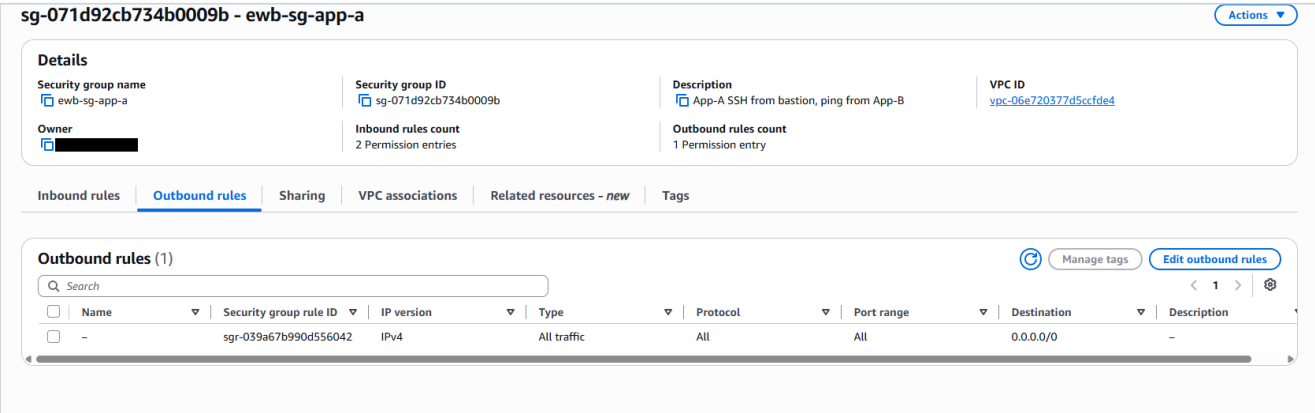
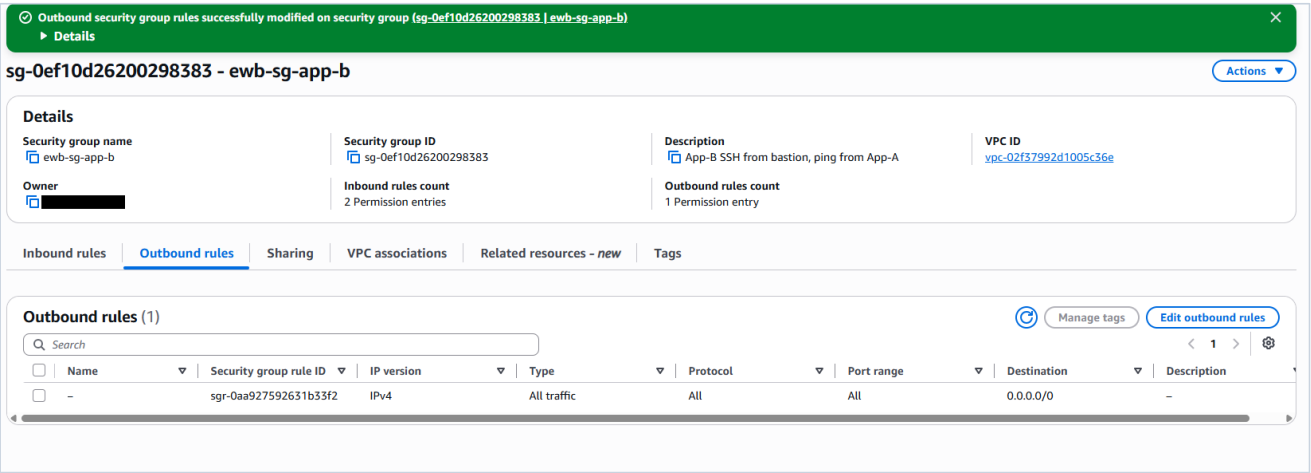


Figure 18. Outbound rules: ewb-sg-app-b

Host / console: VPC console / Security groups / ewb-sg-app-b | **Result: All traffic to 0.0.0.0/0**



Instances

Figure 19. All three instances running (Checkpoint 02)

Host / console: EC2 console / Instances | **Result: 3 of 3 Running, 3/3 checks passed, all in ap-south-1a**

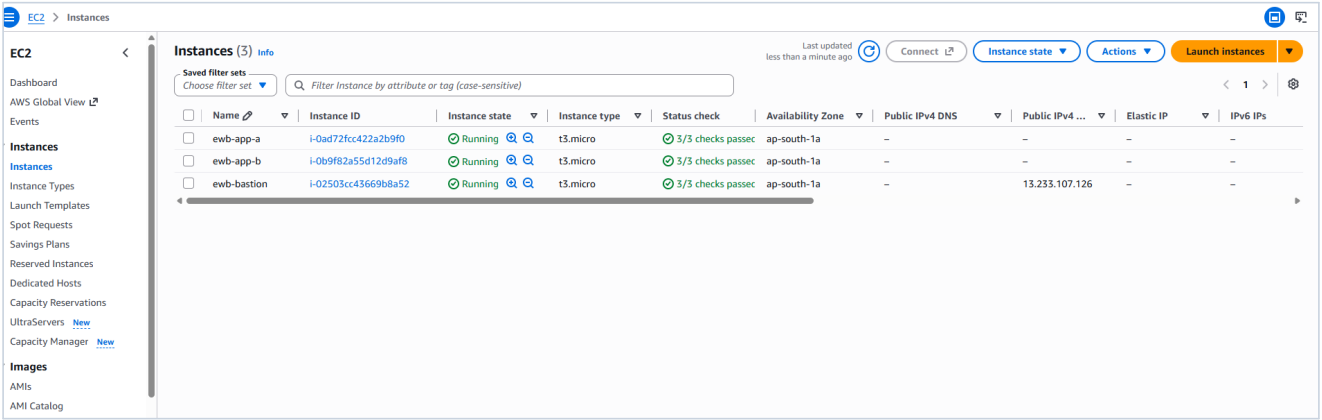


Figure 20. ewb-bastion details

Host / console: EC2 console / ewb-bastion | **Result: Private 10.10.1.10; public IPv4 13.233.107.126; subnet ewb-a-public**

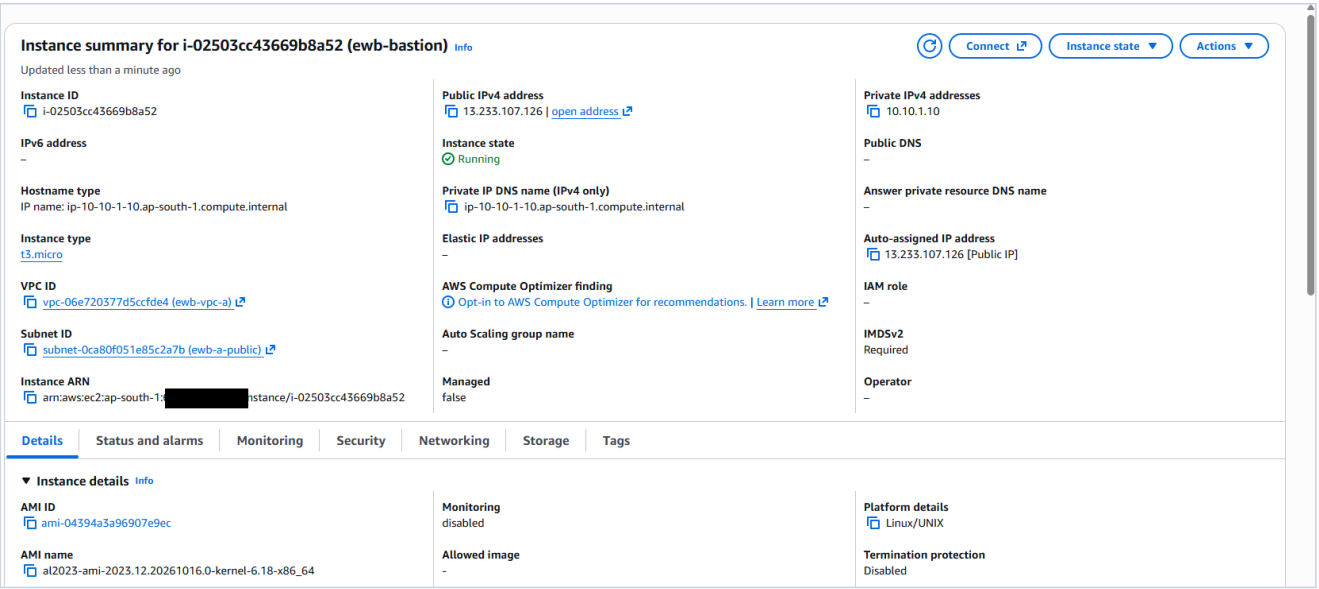


Figure 21. ewb-app-a details - isolation check (Test 05)

Host / console: EC2 console / ewb-app-a | **Result:** **Private 10.10.2.10; Public IPv4 address empty**

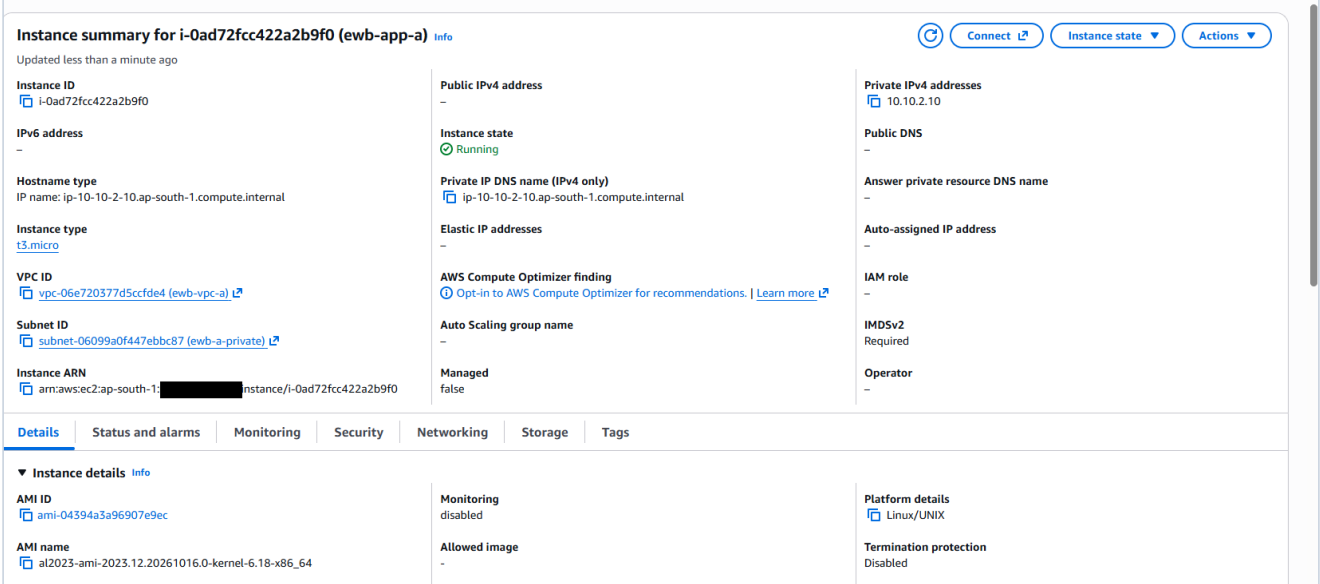
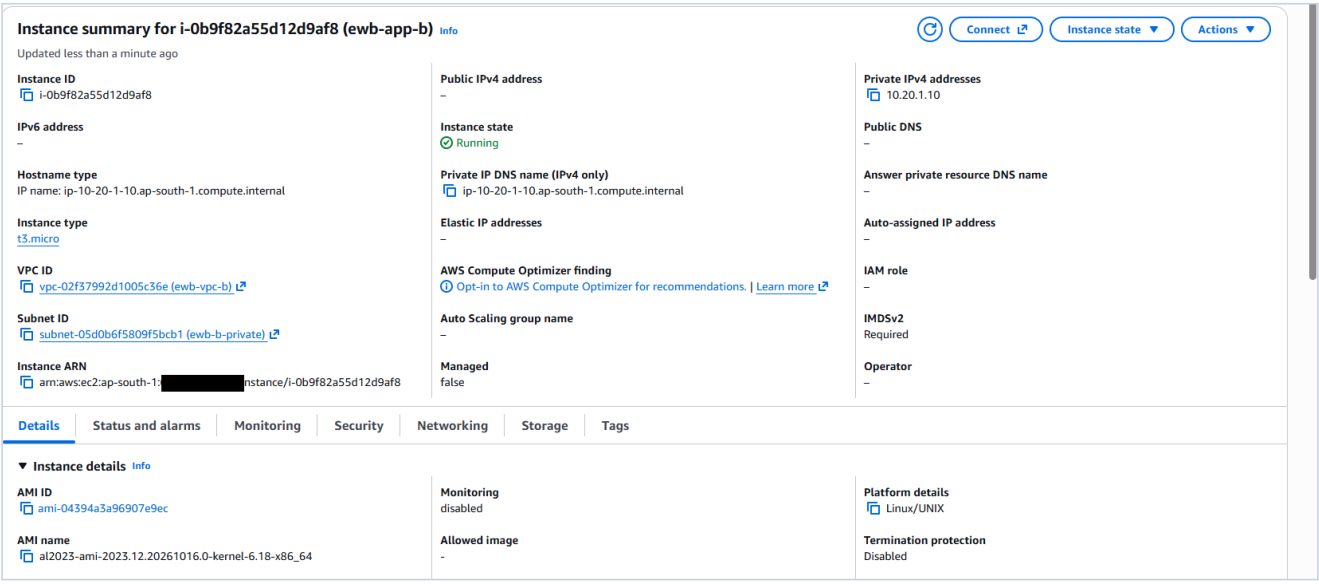


Figure 22. ewb-app-b details - isolation check (Test 05)

Host / console: EC2 console / ewb-app-b | **Result:** **Private 10.20.1.10; Public IPv4 address empty**



4. Administrative access with SSH ProxyJump

The private key stays on the laptop and is never copied to an instance. The laptop reaches the bastion directly; App-A and App-B are reached through the bastion with ProxyJump. This is the local ~/.ssh/config used (the bastion public IP is shown as a placeholder because it was released at cleanup):

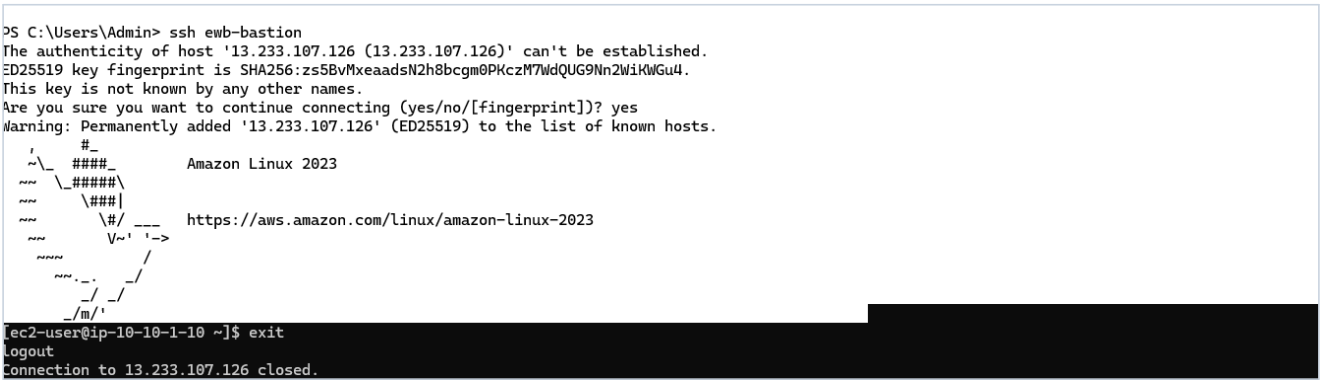
```
Host ewb-bastion
    HostName BASTION_PUBLIC_IP
    User ec2-user
    IdentityFile ~/.ssh/ewb-lab.pem
    IdentitiesOnly yes

Host ewb-app-a
    HostName 10.10.2.10
    User ec2-user
    IdentityFile ~/.ssh/ewb-lab.pem
    IdentitiesOnly yes
    ProxyJump ewb-bastion

Host ewb-app-b
    HostName 10.20.1.10
    User ec2-user
    IdentityFile ~/.ssh/ewb-lab.pem
    IdentitiesOnly yes
    ProxyJump ewb-bastion
```

Figure 23. SSH from the laptop to the bastion

Host / console: Laptop PowerShell: ssh ewb-bastion | **Result:** **Shell prompt ip-10-10-1-10 (the bastion)**

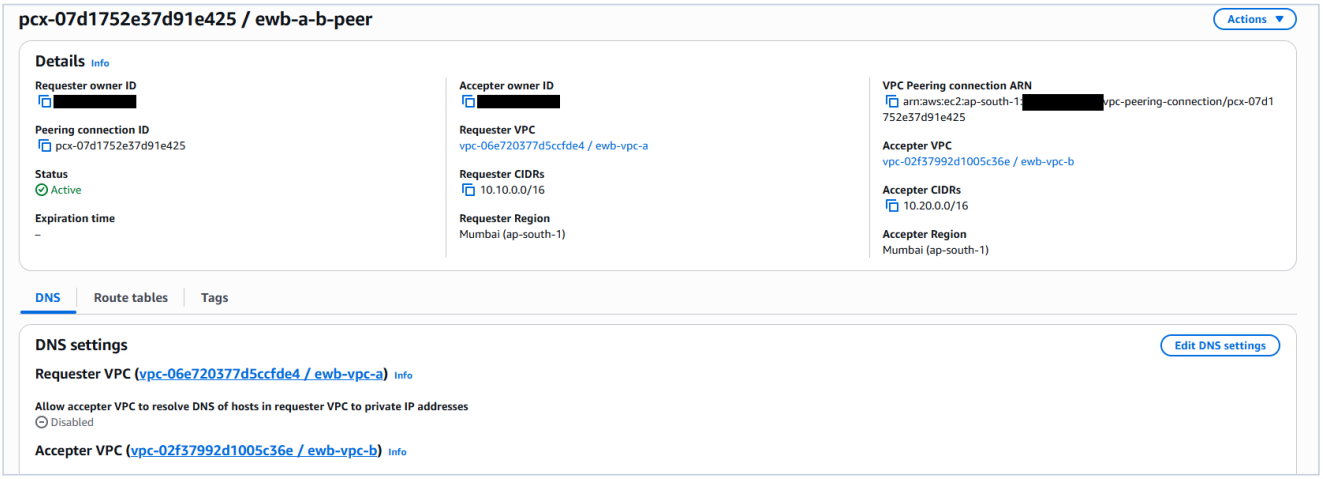


5. Peering connection and routes (Checkpoints 03 and 04)

The peering connection ewb-a-b-peer was requested from VPC A, accepted in VPC B, and became Active. Peering alone does not carry traffic: each side also needs a route to the other VPC's CIDR.

Figure 24. Peering connection ewb-a-b-peer is Active (Checkpoint 03)

Host / console: VPC console / Peering connections | **Result:** **Status Active; requester ewb-vpc-a 10.10.0.0/16; accepter ewb-vpc-b 10.20.0.0/16**



Route table	Associated subnet	Routes added or kept
ewb-rt-a-public	ewb-a-public	10.10.0.0/16 local; 0.0.0.0/0 -> ewb-igw-a; **10.20.0.0/16 -> pcx-07d1752e37d91e425** (bastion SSH to App-B)
ewb-rt-a-private	ewb-a-private	10.10.0.0/16 local; **10.20.0.0/16 -> pcx-07d1752e37d91e425** (App-A to VPC B)
ewb-rt-b-private	ewb-b-private	10.20.0.0/16 local; **10.10.0.0/16 -> pcx-07d1752e37d91e425** (return path to VPC A)

Figure 25. ewb-rt-a-public after peering (Checkpoint 04)

Host / console: VPC console / ewb-rt-a-public | **Result:** **Three routes, all Active, including 10.20.0.0/16 to the peering connection**

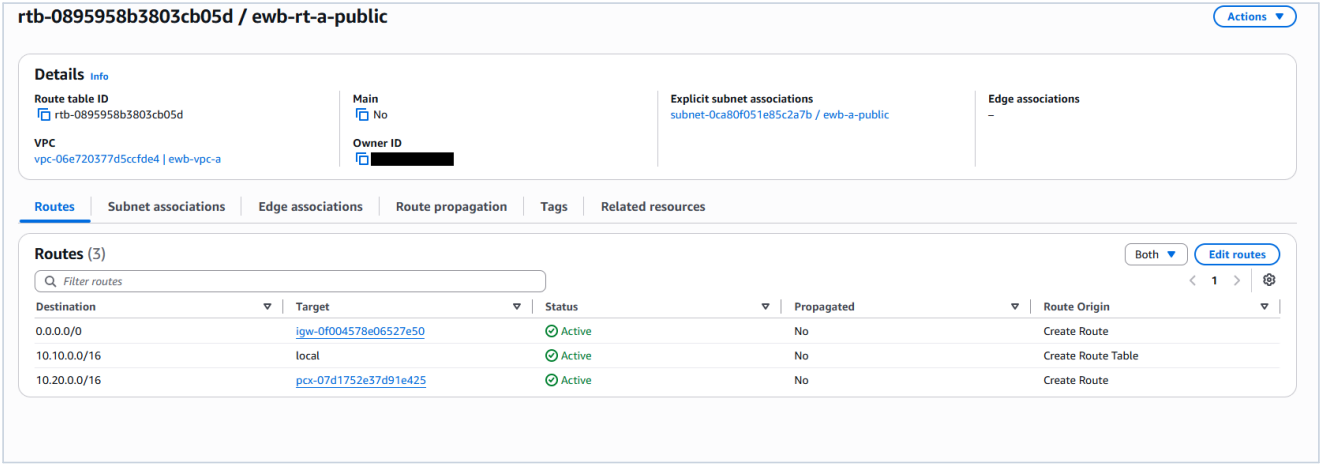


Figure 26. ewb-rt-a-private after peering (Checkpoint 04)

Host / console: VPC console / ewb-rt-a-private | **Result:** **10.20.0.0/16 to the peering connection, Active**

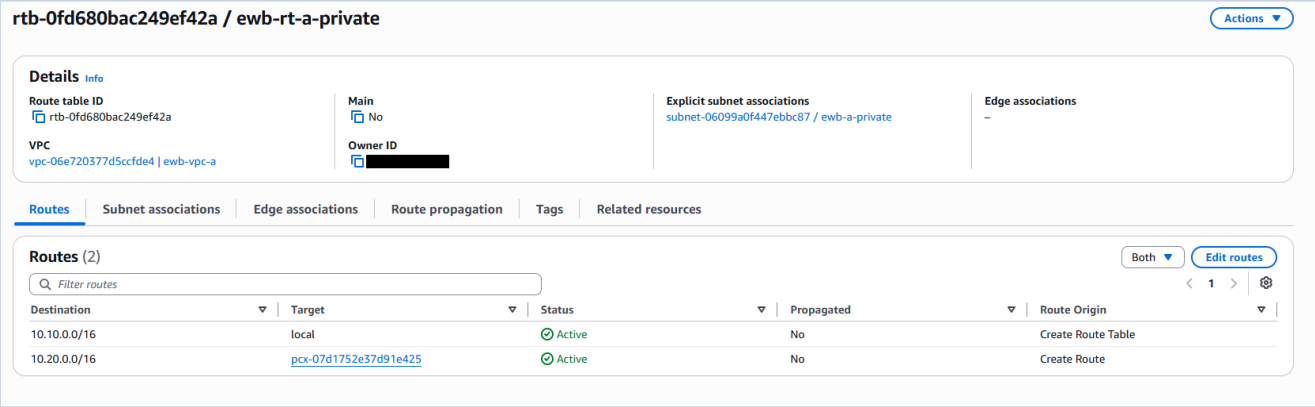
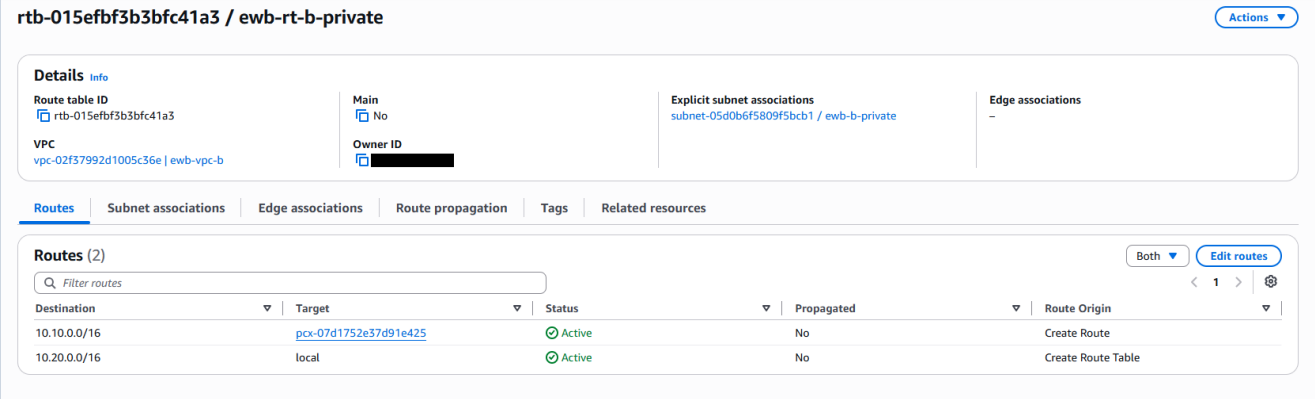


Figure 27. ewb-rt-b-private after peering (Checkpoint 04)

Host / console: VPC console / ewb-rt-b-private | **Result:** **10.10.0.0/16 return route to the peering connection, Active**



6. Verification tests

Every screenshot below is real terminal or console output from the lab. The ping destination is always a private IP.

Test	Where / command	Expected	Observed
01 Baseline	App-A, before peering: ping -c 4 10.20.1.10	No replies	4 sent, 0 received, 100% packet loss
02 Forward path	App-A, after routes: ping -c 4 10.20.1.10	4 replies, 0% loss	4 received, 0% loss, avg 0.250 ms
03 Reverse path	App-B, after routes: ping -c 4 10.10.2.10	4 replies, 0% loss	4 received, 0% loss, avg 0.332 ms
04 Private SSH	Laptop: ssh ewb-app-b	Shell on App-B via bastion	Shell prompt ip-10-20-1-10
05 Isolation	EC2 details: App-A and App-B	No public IPv4	Public IPv4 address empty on both
06 Controlled failure	ICMP rule removed from ewb-sg-app-b; ping from App-A	Replies stop	100% packet loss
07 Recovery	Exact rule restored; ping again	Replies resume	4 received, 0% loss, avg 0.184 ms

Test 01 - Baseline failure (before peering)

Figure 28. Test 01: App-A cannot reach App-B before peering

Host / console: ewb-app-a (10.10.2.10): ping -c 4 10.20.1.10 | **Result:** **PASS - 100% packet loss, as expected without peering or routes**

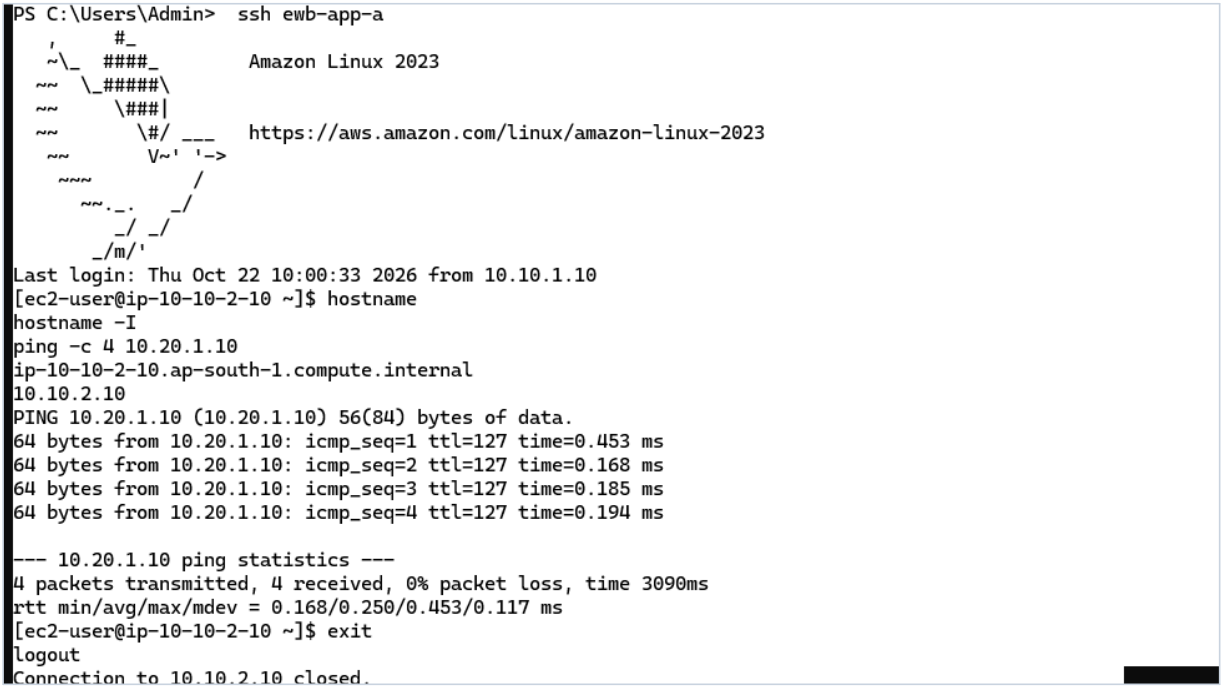
```
PS C:\Users\Admin> ssh ewb-app-a
The authenticity of host '10.10.2.10 (<no hostip for proxy command>)' can't be established.
ED25519 key fingerprint is SHA256:k/vc0S925O5QqTnbUzOMidIqaGTWDl8EosquQlVqv2E.
This key is not known by any other names.
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added '10.10.2.10' (ED25519) to the list of known hosts.
   ,     #_
   ~\_  ####_        Amazon Linux 2023
  ~~  \_#####\
  ~~     \###|
  ~~       \#/ ___   https://aws.amazon.com/linux/amazon-linux-2023
   ~~       V~' '->
    ~~~         /
      ~~._.   _/
         _/ _/
       _/m/'
[ec2-user@ip-10-10-2-10 ~]$ hostname
hostname -I
ping -c 4 10.20.1.10
ip-10-10-2-10.ap-south-1.compute.internal
10.10.2.10
PING 10.20.1.10 (10.20.1.10) 56(84) bytes of data.

--- 10.20.1.10 ping statistics ---
4 packets transmitted, 0 received, 100% packet loss, time 3108ms
```

Test 02 - Forward path (after peering and routes)

Figure 29. Test 02: App-A pings App-B

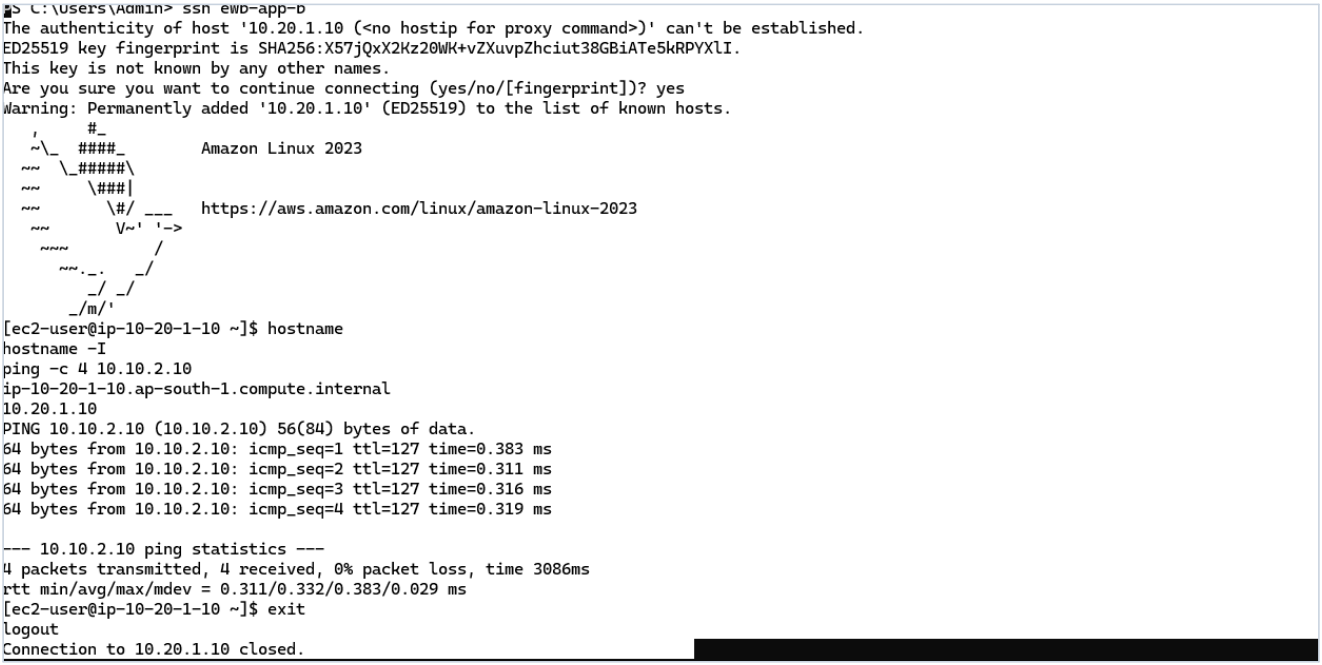
Host / console: ewb-app-a (10.10.2.10): ping -c 4 10.20.1.10 | **Result: PASS - 4 packets received, 0% packet loss**



Tests 03 and 04 - Reverse path and private SSH

Figure 30. Tests 03 and 04: SSH to App-B through the bastion, then ping App-A

Host / console: Laptop: ssh ewb-app-b, then on ewb-app-b (10.20.1.10): ping -c 4 10.10.2.10 | **Result: PASS - shell on App-B reached via bastion; 4 received, 0% packet loss**



Test 05 - Isolation

Figures 21 and 22 (section 3) show that neither ewb-app-a nor ewb-app-b has a public IPv4 address; both are reachable only through the bastion and the peering connection.

Tests 06 and 07 - Controlled failure and recovery

Only the Echo Request rule on ewb-sg-app-b (source 10.10.2.10/32) was removed; the SSH rule was kept. The peering connection and both route tables were unchanged. After the rule was restored exactly (Custom ICMP, Echo Request, 10.10.2.10/32), the ping worked again.

Figure 31. Test 06: ping fails with the ICMP rule removed

Host / console: ewb-app-a (10.10.2.10): ping -c 4 10.20.1.10 | **Result:** **PASS - 100% packet loss while the rule is missing**

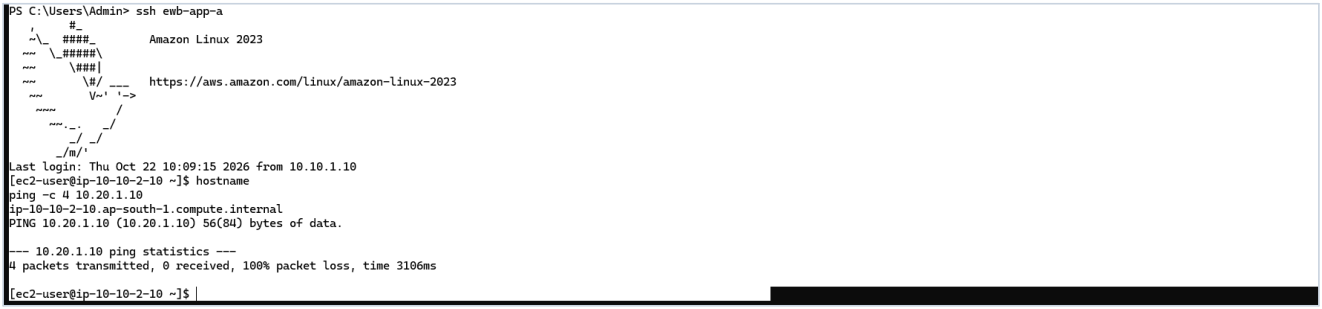


Figure 32. Test 07: ping recovers after the rule is restored

Host / console: ewb-app-a (10.10.2.10): ping -c 4 10.20.1.10 | **Result:** **PASS - 4 packets received, 0% packet loss**

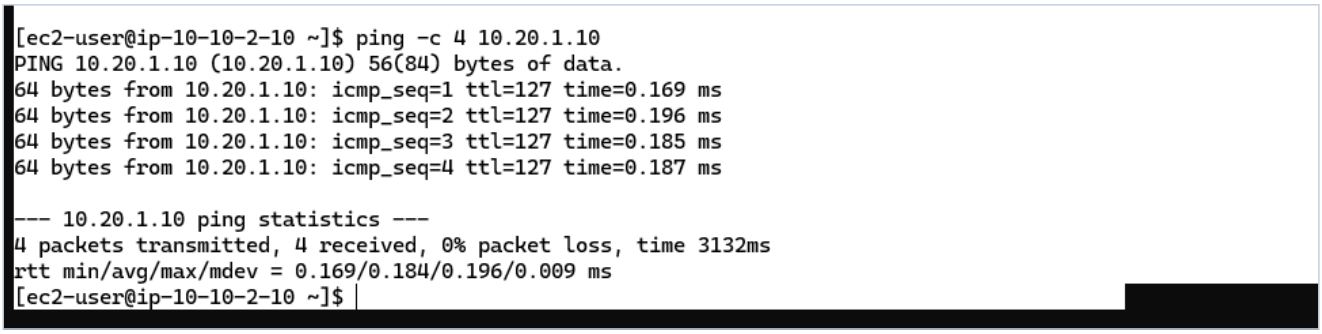
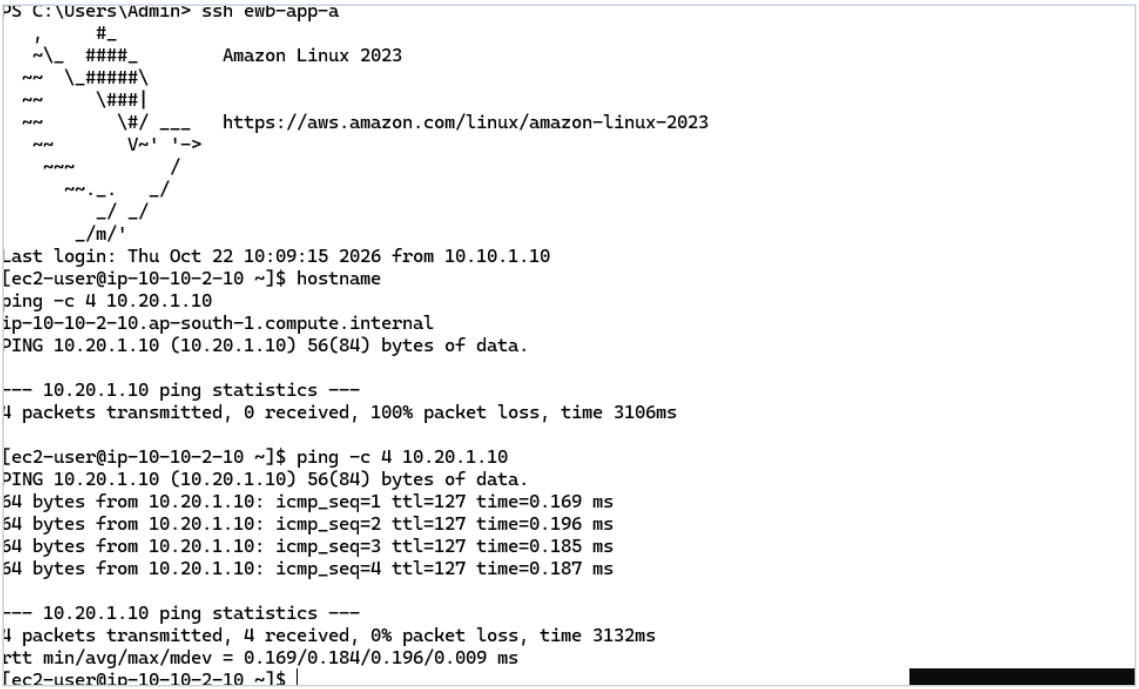


Figure 33. Tests 06 and 07 together in one session

Host / console: ewb-app-a (10.10.2.10): the same ping before and after restoring the rule | **Result:** **Failure followed by recovery**



7. Explanations

Forward path and return path

Forward path (App-A to App-B). A ping from 10.10.2.10 to 10.20.1.10 leaves App-A. The subnet's route table ewb-rt-a-private matches 10.20.0.0/16 and sends the packet to the peering connection. In VPC B the packet reaches 10.20.1.10 only if ewb-sg-app-b allows ICMP Echo Request from 10.10.2.10/32, which it does.

Return path (App-B to App-A). App-B's reply goes from 10.20.1.10 to 10.10.2.10. ewb-rt-b-private matches 10.10.0.0/16 and sends it back through the peering connection. Security groups are stateful, so the reply is allowed automatically without an extra inbound rule on App-A.

The four questions

1. Why is the return route needed? Each VPC makes its own routing decision. Without a route to 10.10.0.0/16 in ewb-rt-b-private, App-B would not know where to send its reply, so the ping would fail even though the request arrived. A one-way route is not enough.

2. Why is the App-B SSH source the bastion's private IP? With ProxyJump the laptop connects to the bastion, and the bastion opens a separate SSH connection to App-B from its own private address, 10.10.1.10. App-B therefore sees 10.10.1.10, not the laptop's public IP. The public subnet also needs the 10.20.0.0/16 peering route so the bastion can reach VPC B.

3. Why does Active peering not guarantee a successful ping? Active only means the link exists. Traffic also needs routes in both directions and security group rules that allow the protocol (and, if customised, network ACLs). The controlled failure proves it: peering and routes stayed intact, yet the ping failed when one security group rule was removed.

4. Can VPC B use VPC A's internet gateway over peering? No. Peering is one-to-one and non-transitive, and it does not forward traffic to a peer's internet gateway. ewb-vpc-b has no internet gateway (0 network connections in its resource map), and App-B has no public IP.

Controlled failure: what it shows

Routing and security groups are separate layers and both must allow the traffic. Removing only the ICMP rule on the destination stopped the ping, restoring the exact rule fixed it, and SSH kept working throughout because its rule was left in place.

8. Cleanup evidence

Resources were removed in reverse dependency order on 26 October 2026. The default VPC and its default resources were left alone.

Step	Action	Result
1	Terminated ewb-bastion, ewb-app-a, ewb-app-b	All three Terminated
2	Deleted ewb-a-b-peer with related routes	Status Deleted; 3 of 3 related routes deleted
3	Deleted ewb-sg-bastion, ewb-sg-app-a, ewb-sg-app-b	3 security groups deleted; only default groups remain
4	Detached and deleted ewb-igw-a	No internet gateways in the Region
5	Deleted the three lab subnets	No subnets found
6	Deleted ewb-rt-a-public, ewb-rt-a-private, ewb-rt-b-private	Only main route tables remain
7	Deleted ewb-vpc-a and ewb-vpc-b	Only the default VPC (172.31.0.0/16) remains
8	Checked volumes and Elastic IPs	No volumes; no Elastic IP addresses
9	Deleted key pair ewb-lab and the local .pem file	No key pairs; no .pem files in ~/.ssh

Figure 34. Cleanup 1: instances terminated

Host / console: EC2 console / Instances | **Result:** **i-0ad72fcc422a2b9f0, i-0b9f82a55d12d9af8, i-02503cc43669b8a52 all Terminated**

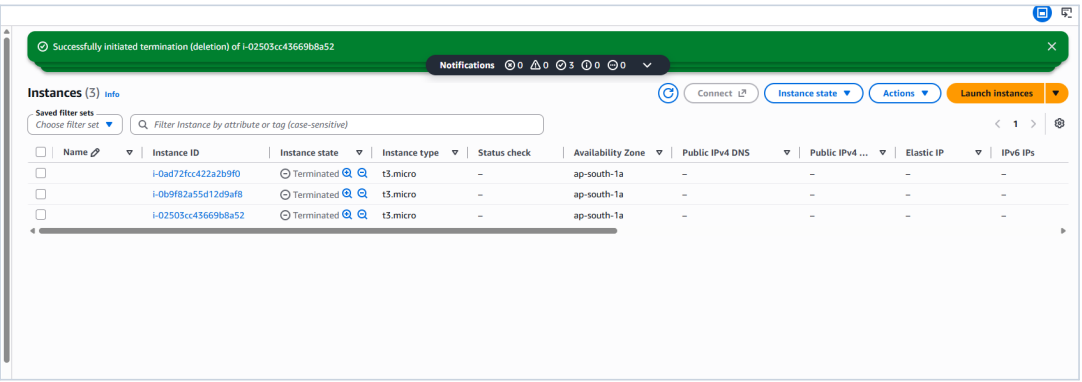


Figure 35. Cleanup 2: peering connection deleted

Host / console: VPC console / Peering connections | **Result:** **pcx-07d1752e37d91e425 deleted; 3 of 3 related routes deleted**

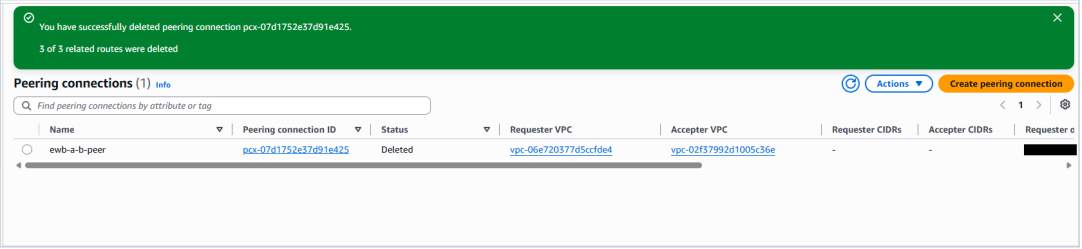


Figure 36. Cleanup 3: security groups deleted

Host / console: VPC console / Security groups | **Result:** **Successfully deleted 3 security groups; only default groups remain**

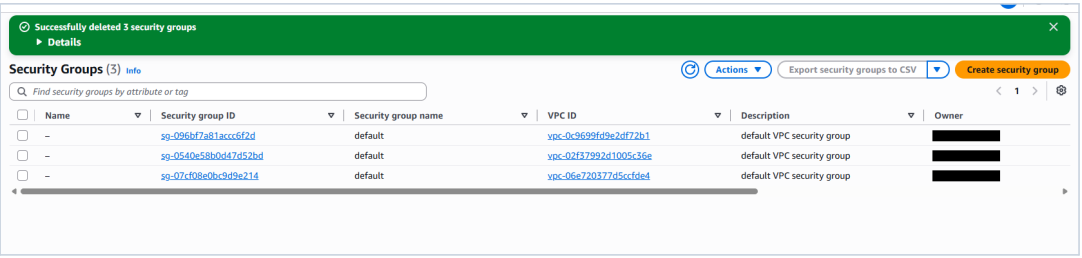


Figure 37. Cleanup 4: internet gateway deleted

Host / console: VPC console / Internet gateways | **Result:** **igw-0f004578e06527e50 deleted; none left in the Region**

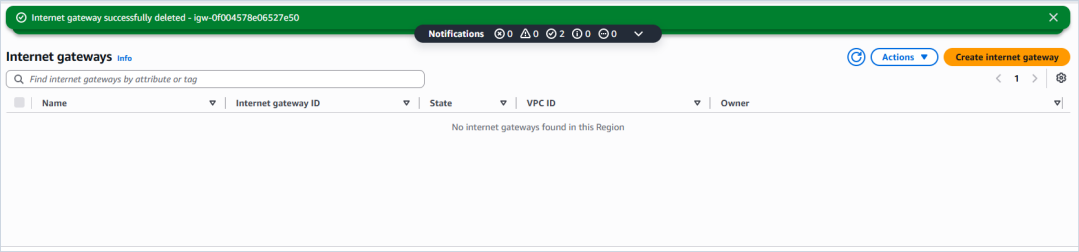


Figure 38. Cleanup 5: subnets deleted

Host / console: VPC console / Subnets | **Result:** **subnet-0ca80f051e85c2a7b, subnet-06099a0f447ebbc87, subnet-05d0b6f5809f5bcb1 deleted**

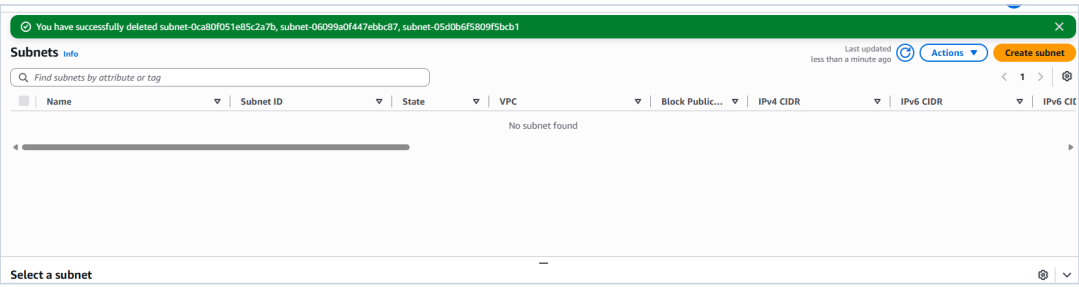


Figure 39. Cleanup 6: custom route tables deleted

Host / console: VPC console / Route tables | **Result:** **rtb-015efbf3b3bfc41a3, rtb-0895958b3803cb05d, rtb-0fd680bac249ef42a deleted; only main route tables remain**

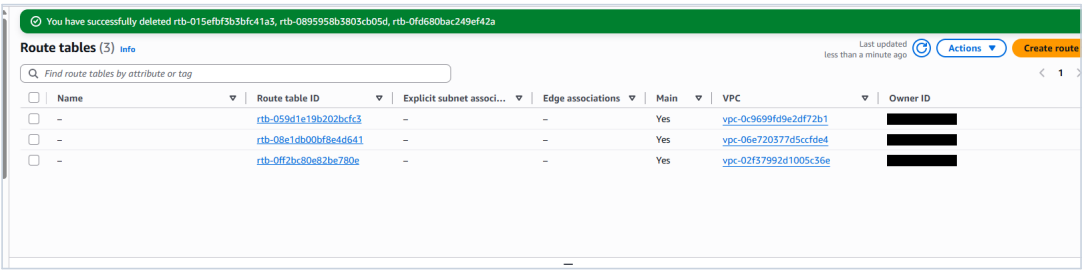


Figure 40. Cleanup 7: lab VPCs deleted

Host / console: VPC console / Your VPCs | **Result:** **Only the default VPC 172.31.0.0/16 remains**

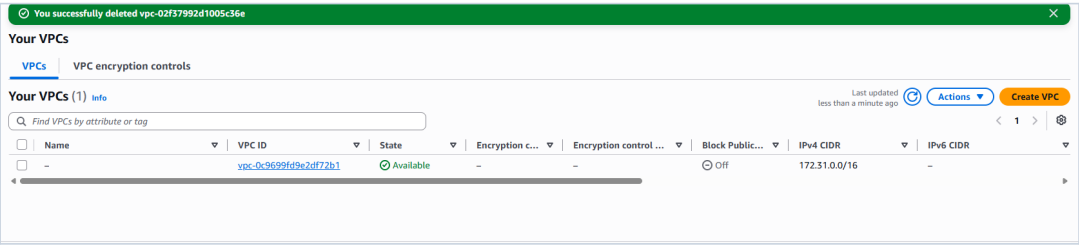


Figure 41. Cleanup 8a: no EBS volumes left

Host / console: EC2 console / Volumes | **Result:** **No volumes in this Region**

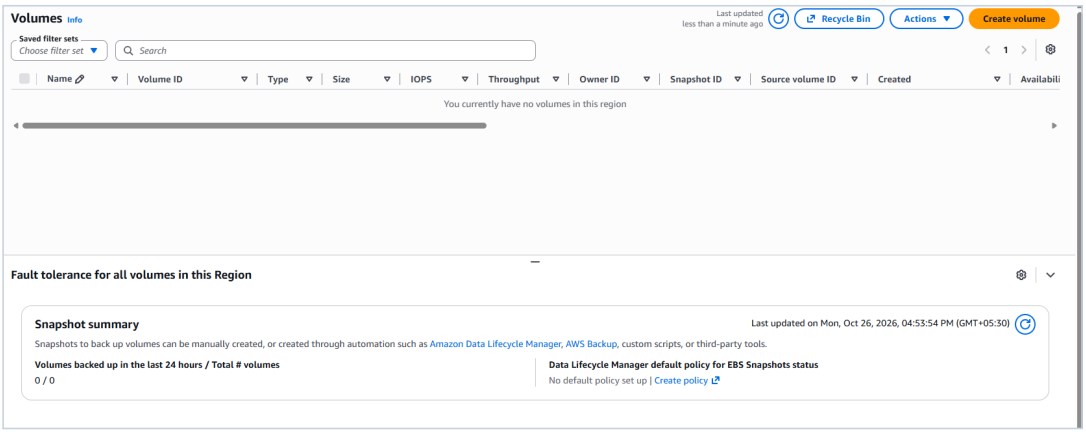


Figure 42. Cleanup 8b: no Elastic IP addresses

Host / console: EC2 console / Elastic IPs | **Result:** **No Elastic IP addresses in this Region**

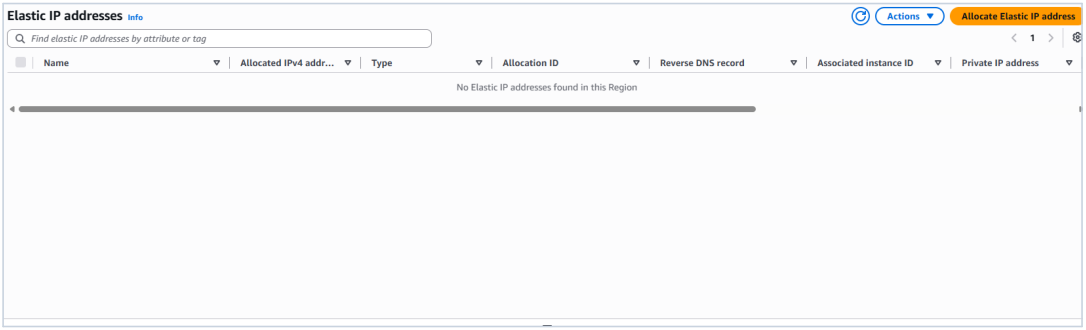
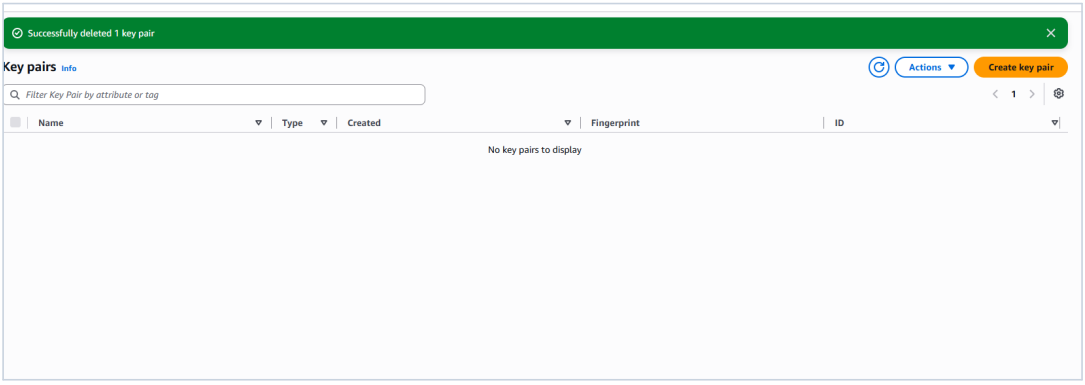


Figure 43. Cleanup 9: key pair deleted

Host / console: EC2 console / Key pairs | **Result:** **Successfully deleted 1 key pair; none left**



9. What I learned

- **CIDR planning comes first.** The two VPC ranges must not overlap for peering to be created, and a subnet CIDR is only valid inside the VPC it belongs to, so choosing the right VPC in the console matters.
- **A subnet is public because of its route table, not its name.** The bastion was unreachable until ewb-a-public was explicitly associated with ewb-rt-a-public (which holds the 0.0.0.0/0 route to the internet gateway).
- **The security group must be attached to the instance.** The bastion first launched with the VPC's default security group, so my SSH rule in ewb-sg-bastion had no effect. Swapping the attached group fixed the timeouts.
- **Routes and security groups are separate layers.** Peering needs a route on both sides and matching security group rules. The controlled failure showed this clearly: removing one ICMP rule broke the ping while routes and peering were untouched.
- **Follow the specification exactly.** I corrected ewb-sg-app-b's outbound rule back to the default allow-all, and recreated ewb-b-private in ap-south-1a (it first landed in ap-south-1c) so all three hosts share one Availability Zone.
- **Check the Region.** At one point the console showed no resources because it was set to a different Region; the lab resources were all in Mumbai.
- **Clean up in dependency order.** Instances first, then the peering connection, security groups, internet gateway, subnets, route tables and VPCs, and finally volumes, Elastic IPs and the key pair.
- **Keep the key private.** The key stayed on my laptop; the bastion with ProxyJump gave access to private hosts without copying it.

Declaration

All screenshots and outputs in this report are from my own execution of the lab. No private keys, passwords or access tokens are included.